



Multi-objective aircraft landing problem: a multi-population solution based on non-dominated sorting genetic algorithm-II

Kimia Shirini¹ · Hadi S. Aghdasi¹ · Saeed Saeedvand²

Accepted: 29 July 2024 / Published online: 9 August 2024

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

The aircraft landing problem (ALP) is a challenging scheduling and optimization problem in the industry and engineering, which has attracted attention in recent decades. Existing research has predominantly concentrated on optimizing aircraft delay and the financial implications of early or late landings. However, given the paramount significance of airport fuel costs at airports and the critical need for efficient fuel utilization, we aim to minimize airplane fuel consumption by streamlining operational time. In this paper, we present an innovative model with two main objectives: minimizing airplane fuel consumption by reducing dwell time and minimizing cost operation. To address these dual objectives concurrently, we propose a new method known as the multi populations of multiple objectives (MPMO) framework, which is modeled through a non-dominated sorting genetic algorithm-II (NSGA-II) called MPNSGA-II. First, MPNSGA-II employs two separate populations to optimize each objective. Second, to prevent populations from fixating solely on their respective single objectives, MPNSGA-II introduces an archive sharing strategy (ASS). This technique stores elite solutions gathered from two populations. Additionally, we introduce an archive update strategy (AUS) to enhance the quality of solutions stored in the archive. The proposed algorithm has been compared with other well-known algorithms, NSGA-II, multi-objective particle swarm optimization (MOPSO), and NSGA-III. The proposed algorithm shows a cost reduction in 18.01%, 16.75%, and 15.21%. Statistical precision, underscored through the application of the nonparametric Friedman test, corroborates the supremacy of the proposed method, clinching the highest ranking compared to state-of-the-art methods.

Keywords Aircraft landing problem · Multi-objective optimization · Multiple populations · NSGA-II

1 Introduction

The rapid progress of technology has significantly propelled the growth of the aviation sector, resulting in a notable surge in the number of flights in recent times [1]. This increased flight activity has given rise to global air traffic challenges, manifesting as issues like heightened aircraft fuel consumption and a rise in flight delays [2]. For instance, according to a report from the Federal Aviation Administration, the aviation industry is anticipated to experience a growth of 150–250% in the future. Additionally, the Airports Council International (ACI) reported that approximately 6 billion passengers were transported by airplanes in 2012, a number expected to double to 12 billion by 2025 [3]. The escalating volume of flights contributes to air traffic congestion, a major factor in delays that have significant repercussions on airport costs. In the USA alone, the annual penalty incurred due to air delays amounts to approximately \$5.5 billion [4].

Scheduling aircraft landings has emerged as a formidable contemporary challenge [5, 6]. The aircraft landing problem (ALP) revolves around efficiently allocating runway space to aircraft for optimal landing times, ensuring they meet their objectives while adhering to constraints. Various algorithms have been employed to address ALP [6]. The CPLEX method is employed for smaller-scale problems [7], whereas heuristic and metaheuristic algorithms like genetic algorithms [8], ant [9] and bee algorithms [10], etc., are deployed for larger-scale ALP instances. Furthermore, researchers have pursued various objectives when tackling ALP [6].

Some studies have aimed at minimizing overall costs [11–18], and [19] involve the minimization of weighted earliness and tardiness, representing the deviation between the actual landing time and the target time. The cost function can take diverse forms, including linear, piecewise linear, or nonlinear functions. Other studies focused on reducing total or average delays for arriving aircraft [20–23], emphasizing cost reduction for airlines. Some research efforts concentrated on minimizing the scheduled time for the last aircraft [24] and [25]. Earlier studies simultaneously considered multiple objectives. For example, [26] concurrently tackled total scheduled time and total costs, while [27] proposed a multi-objective optimization approach targeting both total flight time and the number of sequence changes.

The goals outlined here represent the priorities of airlines, airports, governmental bodies, and entities such as ACI. Airlines are primarily focused on managing operational expenses, while airports heavily depend on revenue generated from passengers. Governments aim to address air pollution and minimize environmental effects [11]. In this study, we aim to take into account both governmental and environmental concerns, all while recognizing the advantages for ACI, which include cost reduction and streamlined operations leading to decreased fuel consumption. To address governmental concerns, we ensured compliance with aviation regulations by maintaining required separation times between aircraft, thus adhering to safety standards. Environmentally, our model prioritizes fuel-efficient landing sequences to reduce carbon emissions. Specifically, we incorporated constraints that minimize holding patterns and unnecessary fuel burn, reducing

the overall environmental footprint of landing operations. When an aircraft can land at the appointed time, it avoids excessive fuel consumption and the resultant air pollution that occurs due to delays. Our optimization model achieves this by scheduling landings in a way that minimizes holding times and optimizes descent paths, thereby ensuring that aircraft do not have to circle the airport or enter holding patterns, which significantly increases fuel use and emissions. By implementing these measures, our approach ensures compliance with governmental regulations while also promoting environmental sustainability. ALP encapsulates the diverse interests of these entities, occasionally resulting in conflicting objectives. Notably, prior research often neglected the considerations of government priorities and environmental impacts. In this study, we aim to take into account both governmental and environmental concerns, all while recognizing the advantages for ACI, which include cost reduction and streamlined operations leading to decreased fuel consumption [28].

The complexity of the ALP is widely recognized as NP-Hard [16], indicating that its computational difficulty increases exponentially as the number of planes involved grows. Traditional exact algorithms frequently struggle to solve large-scale instances of the problem. Therefore, we resort to employing multi-objective metaheuristic algorithms to address this formidable challenge [29]. The objectives of our study—minimizing fuel consumption and costs—are critical concerns in real-world airport operations. Efficiently managing aircraft landings to reduce fuel burn not only leads to cost savings for airlines, but also contributes to environmental sustainability by lowering carbon emissions. This section discusses how our objectives align with operational practices and constraints at airports and provides examples from actual airport operations. Airports and airlines continuously seek to optimize landing sequences to minimize holding patterns and delays, which directly impacts fuel consumption. By efficiently scheduling landings within the defined time windows and maintaining necessary separation times, our approach reduces the need for aircraft to circle or hold, thereby conserving fuel. For example at Hartsfield-Jackson Atlanta International Airport, one of the busiest airports in the world, air traffic controllers use advanced scheduling algorithms to minimize delays and optimize landing sequences, resulting in significant fuel savings for airlines [30].

Airports must ensure that aircraft land within their allocated time slots to maintain operational efficiency and adhere to slot coordination regulations, especially at congested airports. Our model's consideration of earliest landing time (ELT) and latest landing time (LLT) ensures that aircraft land within these slots, aligning with real-world practices. London Heathrow Airport operates under strict slot coordination rules, where each aircraft is assigned a specific landing window. By optimizing landings within these windows, Heathrow manages to accommodate a high volume of traffic efficiently. By aligning our objectives with real-world operational practices and constraints, and providing concrete examples and case studies, we demonstrate the practical relevance and applicability of our research. These enhancements address the reviewer's comments and strengthen the contribution of our study to both academic research and real-world airport operations.

Our research delves into a multifaceted optimization problem that revolves around two crucial objectives. The firstly aim is to minimize aircraft landing time

by incentivizing early landings, while the secondary objective focuses on cost minimization, presenting a conflicting scenario with the first goal. Striking an optimal balance between these two objectives poses a challenging and intricate task, and our study addresses this challenge to achieve an equilibrium between operational efficiency and cost-effectiveness within the constraints of air traffic management. To tackle this ALP with multiple criteria, we propose a multi-objective optimization approach based on the Non-dominated Sorting Genetic Algorithm-II (NSGA-II). A new approach, named MPNSGA-II, is introduced to address the challenge of optimizing two objectives simultaneously within a framework called multiple populations for multiple objectives (MPMO). Initially, MPNSGA-II employs separate populations to optimize each objective. To prevent populations from solely focusing on their respective objectives, an archive sharing strategy (ASS) is introduced. This strategy stores elite solutions from both populations in an archive, enabling each population to gain optimization insights from the other objectives. Consequently, MPNSGA-II can effectively approximate various segments of the entire Pareto front (PF). Additionally, an Archive Update Strategy (AUS) is proposed to enhance the quality of solutions within the archive. Furthermore, our proposed methodology exhibits improved convergence capabilities in comparison with the previously discussed approaches. We used the inverted generational distance (IGD) metric to evaluate convergence, and a lower value of IGD indicates greater convergence. To evaluate the diversity, we used the hypervolume (HV) metric, where a higher HV value indicates that the algorithm is more effective and that the Pareto front point diversity is greater. Statistical analyses demonstrate that the proposed method achieves the highest ranking in all of the tests. In the following, we describe the principal contributions of this research study:

- 1) **Development of a Novel Multi-Population Approach:** We introduce a novel multi-population method based on the Non-dominated Sorting Genetic Algorithm-II (NSGA-II) for solving the Multi-Objective Aircraft Landing Problem (MOALP). This method effectively optimizes aircraft landing schedules to balance operational efficiency and environmental impact.
- 2) **Demonstration of Methodological Success:** We demonstrate the effectiveness and robustness of our proposed method through extensive simulations and comparative analyses. Specifically, our approach achieves superior performance in terms of minimizing operational delays, reducing fuel consumption, and enhancing overall airport efficiency. This validation underscores the practical applicability and efficiency of our proposed approach in real-world air traffic management scenarios.

The continuation of the article is divided as follows. Section 2 includes the explanation of the airplane landing problem, which includes the definition of the problem, the objectives of the problem, the limitations of the problem, and finally, the formulation of it. In Sect. 3, we present the algorithm designed for the problem. In Sect. 4, the results of the algorithm are reviewed, and in Sect. 5, summarizing and pointing to future works are discussed.

2 Related work

The ALP is a critical area of aviation research, aiming to optimize the process of safely guiding an aircraft to the ground. This literature review provides an overview of key research findings, methodologies, and challenges in addressing ALP.

Since the 1990s, researchers have diligently tackled the ALP due to its increasing complexity and real-world implications. Apart from exact methods, heuristic approaches have gained attention for efficiently searching for acceptable solutions to this NP-hard problem. While exact methods offer optimal solutions, their computational time escalates exponentially with problem size, limiting their applicability to small or medium-scale problems [31].

In recent years, a variety of methods have been proposed to address the aircraft landing scheduling problem, broadly categorized into dynamic programming, mixed integer methods, heuristic methods, and metaheuristic methods.

Dynamic programming methods have been instrumental in solving specific instances of the problem. Bernell et al. [32] delve into the planning of aircraft landings on a runway, considering various constraints such as time windows for each aircraft's landing and minimum separation times between consecutive landings, which are contingent upon the weight classes of the aircraft involved. They formulate a multi-objective function encompassing factors like runway throughput, timeliness of arrivals, lateness, and fuel costs associated with aircraft maneuvers and additional flight time needed to adhere to the landing schedule. This study explores both static scenarios, where the details of incoming flights are known beforehand, and dynamic scenarios, where flight arrival information becomes available progressively. In dynamic scheduling, the algorithm is designed to update the existing schedule to accommodate newly arriving aircraft periodically. The author investigates the application of dynamic programming and the implementation of local search algorithms to address both static and dynamic problems, utilizing random test data as well as real data obtained from London Heathrow Airport.

Faye et al. [33], delve into the challenge of aircraft landing in a multi-band mode scenario, aiming to arrange incoming planes onto available runways at the airport efficiently. The timing of landings falls within predetermined time windows, with strict adherence to safety separation intervals between successive landings. This novel approach offers a methodical solution to the problem, employing an innovative strategy centered on approximating the separation time matrix and time discretization. The study focuses on static problem settings with a singular objective, particularly suited for scenarios with relatively smaller dimensions.

In the second category, we focus on studies employing the mixed integer programming (MIP) method to address pertinent issues. Malik et al. [34] tackled the challenges of terminal airspace and airport congestion management by integrating entry and exit control measures. Their study addressed the multi-band mode, aiming to minimize delay. It involved landing time constraints within predefined windows and enforced safety separation limits between consecutive landings. Additionally, it emphasized the need for horizontal distance maintenance between aircraft landing on different runways.

Pohl et al. [35] introduced an optimization model targeting runway scheduling, particularly in the context of winter operations. Given periodic runway closures for snow clearance during snowfall, their model aimed to simultaneously plan snow removal activities across multiple runways while allocating runways and scheduling aircraft take-offs and landings. Formulated as a mixed integer linear problem, their approach considered the problem across multiple bands and prioritized minimizing delay costs. In general, exact methods like dynamic programming and mixed integer methods face limitations when attempting to solve problems of large scale. Metaheuristic methods have received less attention compared to the aforementioned methods.

Shohel et al. [36] address the challenge of efficiently planning take-off and landing operations within a busy terminal maneuvering area (TMA). Their mathematical formulation considers various aspects simultaneously: accurately predicting aircraft trajectories at each TMA node, modeling safety rules between successive aircraft with high precision, and scheduling to minimize aircraft delay, travel time, and fuel consumption within tight time constraints. Computational experiments were conducted using real-world data from Milano Malpensa. Also, Hammouri et al. [37] propose an efficient method for the aircraft landing problem (ALP) by hybridizing iterative local search (ILS) and simulated annealing (SA) algorithms. Their approach schedules incoming aircraft to land on the runway within predefined time frames, aiming to find scheduling solutions that meet target time constraints. They address the ALP in multi-band mode, focusing on minimizing delay.

The third category discussed heuristic methods [38]. Salehipour et al. [39] planning for the landing time of the plane to minimize the total deviations from the target landing times. This work proposes a new approach to solve the ALP, which includes sequence separation and aircraft scheduling. While obtaining the optimal landing sequence is very difficult, especially for large samples, an optimal program for a given sequence can be provided in polynomial time. They did this problem for the single runway mode.

An examination of existing studies reveals that the incorporation of fuel consumption reduction objectives in the ALP has received limited attention, with the utilization of simplistic criteria to define this objective. In contrast, numerous studies have employed conventional operators of single-objective optimization. Because ALP is a large-scale problem, we attempted to improve the solution of the multi-objective ALP problem through the multi-population framework in this paper, in addition to the more comprehensive definition of the fuel consumption reduction objective and cost reduction.

3 Multi-objective optimization for aircraft landing problem

To enhance the model's realism and its ability to accommodate diverse airport demands, this section proposes a multi-objective aircraft landing problem (ALP) that addresses two key objectives. Specifically, we introduce a novel framework called multi-population for multiple objectives (MPMO) based on NSGA-II, named MPNSGA-II, aimed at simultaneously minimizing fuel consumption and

operational costs. Initially, MPNSGA-II uses two populations to minimize each of the two objectives. Second, an archive sharing strategy (ASS) is proposed to store the elite solutions acquired from the two populations so that the populations can obtain optimization knowledge about the other objectives from the archive, preventing each population from concentrating just on its corresponding single objective. In our study, the term elite solution refers to non-dominated solutions within multi-objective optimization, aligning with Pareto optimality principles. These solutions balance operational efficiency and environmental impact, as they cannot be improved in one objective without sacrificing performance in another. By focusing on non-dominated solutions, we provide decision-makers with a range of high-quality alternatives that consider trade-offs between conflicting goals. In this manner, distinct regions of the whole Pareto front (PF) can be approximated using MPNSGA-II. Third, to further enhance the caliber of the solutions in the archive, an archive update strategy (AUS) is suggested. We will go over each of the operations above in detail in the sections that follow.

3.1 Problem definition

The problem of aircraft landing scheduling includes determining the optimal landing time for landing planes as well as determining their landing order. Suppose P represents a set of planes $p = \{1, 2, \dots, n\}$ that intend to land on the runway. ALP tries to get the order and time of the plane's landing according to the defined goals and limits. In the case of ALP, we try to make the landing time close to the target landing time. There will be a penalty for the airport if the plane arrives early or late compared to the target landing time. Therefore, one of the basic goals is to land at a time close to target landing time (TLT). Figure 1 shows the ALP in summary form. Several planes are moving toward the airport runway and intend to land on the runway. The restrictions of the distance and time window between the planes are being observed. In the following, we will discuss the objectives presented in this article

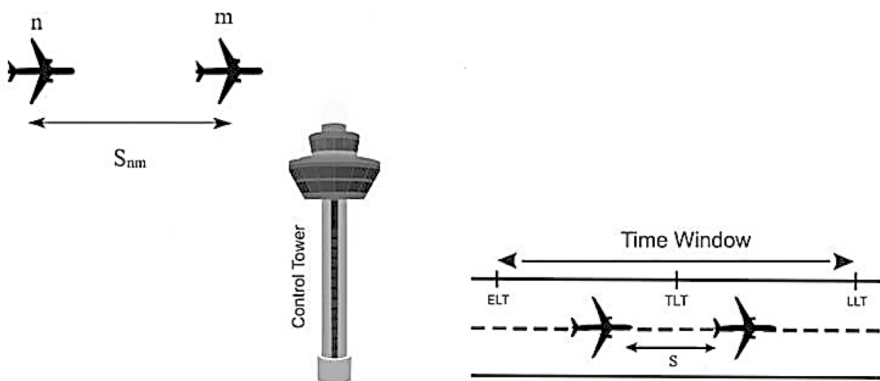


Fig. 1 Aircraft landing problem

and the existing limitations in detail. In the following, we will discuss the parameters mentioned in Fig. 1.

3.2 The constraints

In every optimization problem, several constraints must be met. In this issue, we present two vital restrictions that are very necessary to observe.

3.2.1 Time window

The timing of aircraft landings is contingent upon factors such as their speed, prevailing weather conditions, and fuel consumption rates. Given the dynamic nature of aircraft speed and external conditions, only planes without right-of-way in a predefined timeframe are permitted to initiate their landing procedures. For each aircraft, two fundamental landing parameters are considered:

1. **ELT (Earliest Landing Time):** This represents the earliest permissible moment at which the aircraft is authorized to commence its landing.
2. **LLT (Latest Landing Time):** This signifies the latest allowable time for the aircraft to land.

The time interval between ELT and LLT is referred to as the “Time Window.” This ensures that each aircraft lands within a specific period, accommodating varying conditions such as airspeed and weather. The concept of the time window is illustrated in Fig. 1.

3.2.2 Separation time

To maintain aerodynamic stability and ensure safety, aircraft must maintain a certain distance from each other during landing approaches. This separation is critical to prevent wake turbulence and potential collisions. In our study, we use sequential separation, which imposes the following constraint for any two aircraft i and j :

$$T_i + S_{ij} \leq T_j \text{ if } T_i < T_j.$$

where.

T_i : Target landing time of aircraft i .

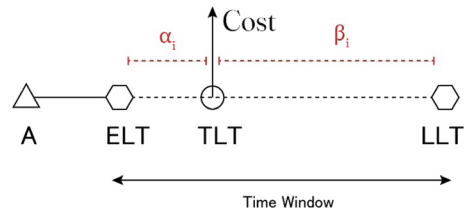
T_j : Target landing time of aircraft j .

S_{ij} : Minimum separation time required between aircraft i and j to maintain safety.

This constraint ensures that the landing sequence respects the necessary separation distances between consecutive aircraft, as shown in Fig. 2.

3.3 Objectives of the presented model

The ALP is a renowned problem in aviation optimization with diverse objectives. Here, our objectives involve two key aspects: first, reducing the costs associated with planes arriving either early or late, and second, cutting down the time required

Fig. 2 Cost objective

for landing (plane landing times) to lessen fuel consumption. The following will provide a more detailed explanation.

3.3.1 Operating cost

One of the most common objectives in the field of ALP is to reduce costs caused by deviation from target landing time. In other words, a penalty is imposed on the airport for each time unit of early arrival or late arrival compared to TLT. Therefore, one of the goals of airports is to reduce costs caused by early arrival or late arrival (delay) in landing time. This goal can include reducing the delay caused by the landing of planes. Equation 1 shows this objective.

$$\min f_1 = \sum_{i=1}^m \alpha_i \times (T_i - x_i) + \beta_i (x_i - T_i) \quad (1)$$

Figure 2 depicts the constraint related to time windows. The illustration makes it evident that as an aircraft deviates further from its TLT, it incurs penalties, which β_i represents the penalty for each unit of delay and α_i represents the penalty for each time unit of early arrival.

3.3.2 Dwell time (fuel consumption)

One of the objectives conspicuously absent in the recent articles is the consideration of the time taken during aircraft landings. Within the available data, we have information on when planes become visible to the control tower. Reducing fuel consumption to mitigate air pollution aligns with the government's interests. Airports aim to have planes land once the control tower spots them. In essence, the first objective seeks to bring the landing time close to the TLT, while the second objective aims for a landing time close to the ELT. To approach TLT, dwell time increases, which, in turn, leads to higher fuel consumption. This approach is advantageous in terms of minimizing aircraft landing time and enhancing overall operational efficiency. However, our study encompasses a second, competing objective, which introduces a trade-off with the time-driven goal. This secondary objective pertains to the cost associated with aircraft landing operations. Regardless of how close the aircraft's arrival time is to the earliest allowable landing time, there is a set of associated penalties. This implies that the closer the aircraft lands to the earliest time, the lower the fines incurred, but the trade-off is that

the cost-related objective conflicts with the primary time-driven objective. Equation 2 is provided to elucidate the second objective of this study.

$$\min f_2 = \sum_{i=1}^m (x_i - A) \quad (2)$$

In Eq. 2, ELT indicates the earliest landing time seen by the control tower and x_i indicates the landing time of the aircraft.

Figure 3 illustrates the second objective emphasized in this paper, which is the reduction in dwell time. Achieving this objective requires the aircraft's landing time to align closely with the moment it becomes visible to the control tower. Simultaneously, it is imperative to adhere to the constraints outlined in the problem. This second objective revolves around the endeavor to land at a time in proximity to the ELT. In contrast, the first objective is oriented toward landing as close as possible to the TLT. Consequently, these two objectives involve a trade-off, where they are in tension with each other.

3.4 Formulation of the presented model

ALP can be expressed in the following mathematical form.

$$F = \text{minimize } (f_1, f_2) \quad (3)$$

Subject to:

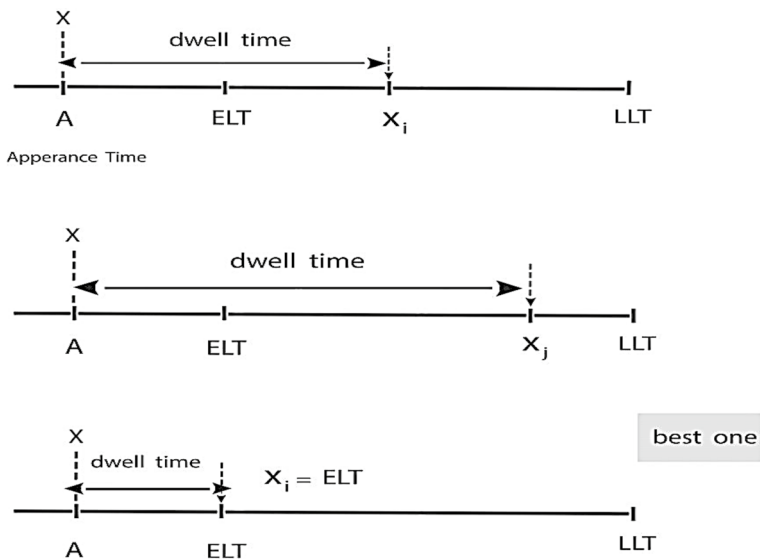


Fig. 3 Dwell time objective

$$ELT_n < x_n < LLT_n; \forall n \in \{1, 2, \dots, P\} \quad (4)$$

$$k_{nm} + k_{mn} = 1; n \neq m \in \{1, 2, \dots, P\} \quad (5)$$

$$x_m \geq x_n + S_{mn}\delta_{nm} - \delta_{mn}(LLT_n - ELT_m); \forall n, m, n \neq m \in \{1, 2, \dots, P\} \quad (6)$$

$$E_m > TLT_m - x_m \quad (7)$$

$$0 \leq E_m < LT_m - ELT_m \quad (8)$$

$$L_m > x_m - TLT_m \quad (9)$$

$$0 \leq L_m < LLT_m - TLT_m \quad (10)$$

$$x_m = TLT_m - E_m + L_m \quad (11)$$

Equation 3 shows the goals of the problem, which include two important goals: reducing operational costs and reducing the time of work. Constraint 4 shows the time window limit, which represents the allowed landing time range of the i -th plane. Constraint 5 makes sure that the scheduled landing times of two different aircrafts are different. In other words, two different planes have different landing times. Formula 6 shows the second limitation of the problem, i.e., the time interval of 2 planes n and m from each other. Constraints 7 to 11 are for defining the earliness and tardiness of landing. Table 1 shows the specific notations used in this problem.

3.5 Solution encoding

One pivotal stage in solving problems with optimization algorithms involves the initial representation of the issue. This article presents an innovative method for

Table 1 Notation and variables for ALP formulas

Notations	variables
P: number of aircraft	x_n : The time of the plane's landing
ELT: Earliest Landing time	$E_m = \max(TLT_m - x_m, 0)$
TLT: Target Landing Time	
LLT: latest Landing Time	
S_{nm} : The separation time of aircraft n, m	$L_m = \max(x_m - TLT_m, 0)$
α_n Penalty for each time unit of early arrival of the plane	$k_{nm} = \begin{cases} 1 & \text{if } n \text{ lands before } m \\ 0 & \text{else} \end{cases}$
β_n Penalty for each time unit of late arrival of the plane	$\delta_{nm} = \{0, 1\}$
	T_i : Target Landing Time of the plane
$K_{nm} = \{0, 1\}$ $n \neq m, [1, 2, 3, \dots, NA]$ Landing sequence	
A : The time of the appearance of the plane	

establishing optimal landing times for aircraft. To encode this problem with precision, chromosomes are employed as arrays, each with a length corresponding to the number of aircraft involved.

The encoding procedure comprises two sequential steps. Firstly, for every chromosome representing an individual aircraft, the determination of landing time commences by generating a random number within the $[0, \text{LLT}-\text{ELT}]$ range. This generated time is subsequently added to the ELT, thereby setting it as the landing time for the corresponding aircraft. Furthermore, in each iteration of the process, the ELT for the planes experiences continual adjustments influenced by the time intervals between them.

Figure 4 depicts a chromosome formed in the initial stage for aircraft1 with ten aircraft, showcasing encoded data vital for determining landing times. This encoding approach acts as the fundamental basis for succeeding phases of the optimization algorithm, streamlining the intricate process of determining aircraft landing times effectively.

The next crucial step involves updating the ELT system of the aircraft by the prescribed separation time limit from the preceding plane. For this purpose, if the landing time of the i -th plane is equal to x_i and the ELT of the $i+1$ th plane is ELT_{i+1} . Then, the new value of ELT of the $i+1$ th aircraft is updated as follows:

$$\text{ELT}_{i+1} = \max\{x_i + S_{i,i+1}, \text{ELT}_{i+1}\}$$

This methodology calculates the landing time of the planes by iteratively updating their ELT values. In our method, a solution (chromosome) is represented by an array of these random keys. Each element in the array corresponds to an aircraft, and its value indicates the proposed landing time. This random key array is then used to determine the sequence and timing of landings.

3.6 MPMO framework

To address challenges related to fitness assignment and diversity maintenance in multi-objective problems (MOPs), Zhan et al. introduced the coevolutionary approach known as MPMO [40]. In this technique, for an M -objective problem, M populations are established, with each population dedicated to concurrently optimizing a distinct objective [41]. As a result, in this paper, two populations are utilized to optimize two objectives individually. The assessment of a solution's

Random key	3	4	11	12	21	12	41	11	14	13
ELT	89	96	110	120	124	126	129	135	160	195
$\text{ALT}[1] = \text{random key} + \text{ELT} \rightarrow \text{ALT}[1] = 3 + 89$										

Fig. 4 Encoding procedure

performance in each population is based on its corresponding optimization objective rather than solely relying on the Pareto dominance rule. It is important to note that genetic algorithm operators, namely the crossover and mutation operators, are employed in each population to generate offspring. NSGA-II is chosen for its suitability in addressing discrete optimization problems. Further elaboration on the NSGA-II operators can be found in Sects. 3.6.2 and 3.6.3. The MPMO framework allows for an in-depth exploration of each objective. To prevent each population from concentrating solely on one objective and to encourage coevolution among different populations, the archive sharing strategy (ASS) is introduced. Detailed insights into ASS are provided in Sect. 3.6.2.

3.6.1 Crossover operator

In this article, the single-point crossover method is used to combine parents. In this one-point crossover, a random cut point is selected, and the characteristics of the two parents are moved from that cut point. Figure 5 is an example of the crossover type of this problem.

3.6.2 Archive sharing strategy

The elite solutions discovered throughout the evolutionary process are stored in the archive, symbolized by the A, which is built in MPMO. Similar to the initial solution, A is initialized arbitrarily. Every solution in population k is chosen as p_1 , while a randomly selected solution in population A is chosen as p_2 . Every time a crossover operation is performed, a random number r between $[0, 1]$ is created. If r is less than or equals the crossover probability, the crossover operator is applied to p_1 and p_2 to produce offspring c_1 and c_2 , and the c_1 or c_2 with the best performance on the k th goal ($k \in [1, 2][1, 2]$) is preserved to create a new population. Otherwise, the better one between p_1 and p_2 is preserved. Algorithm 1 shows the steps mentioned for the ASS algorithm. AST in MPMO not only can retain good characteristics in its corresponding optimization objective, but also can effectively utilize the information on the other objective.

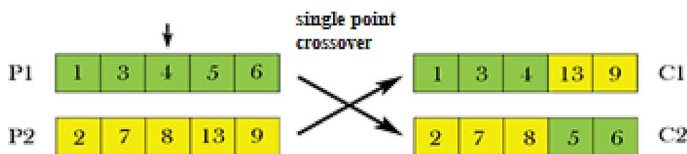


Fig. 5 Single-point crossover method

Algorithm 1 ASS (Pop, A, Pr_c)

Input: Pop_k, A (archive), Pr_c (probability of crossover).

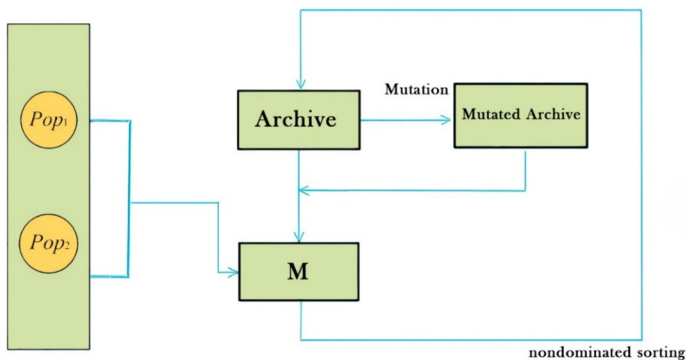
```

1:   For  $k=1$  to 2
2:      $Pop'_k \leftarrow \emptyset$ ;
3:     For  $i=1$  to  $|Pop_k|$ 
4:        $p_1 \leftarrow Pop_k^i$ ;
5:       Randomly pick an elite solution, denoted as "E", from the archive "A".
6:        $p_2 \leftarrow E$ ;
7:        $r = rand(0,1)$ ;
8:       If  $r \leq Pr_c$ 
9:          $(c_1, c_2) = single-point(p_1, p_2)$ ;
10:         $x = \arg \min_{x \in \{p_1, c_1, c_2\}} \{f_k(x)\}$ ;
11:         $Pop'^i_k \leftarrow \{x\}$ ;
12:      Else
13:         $x = \arg \min_{x \in \{p_1, p_2\}} \{f_k(x)\}$ ;
14:         $Pop'^i_k \leftarrow \{x\}$ ;
15:      End If
16:    End For
17:  End For
Output:  $Pop'_k$  (Populations produced through the crossover process.)

```

3.6.3 Archive-update strategy

We suggested that A be updated by the AUS to account for freshly created populations and produce solutions that perform better. In MPNSGA-II, the maximal size of the archive is set to NA (maximum size of the archive). The empty set M is filled up for each generation by adding all solutions from all populations. Then, all solutions in the archive set are added to M. After that all solutions in the archive are mutated and added to M. Then, we delete all duplicate solutions. In the end, the process of non-dominated sorting is called on the set M, to sort all the solutions according to their dominance. Then, we choose the NA's first solutions as the new archive.

**Fig. 6** Archive update strategy

In Algorithm 2, the AUS's pseudocode is displayed. The execution of the archive update technique is shown in Fig. 6.

Algorithm 2 AUS (Pop_k, A, NA)

Input: Pop_k (populations, $k \in [1, 2]$), A (archive), NA (The archive's maximum size).

```

1:    $M \leftarrow \emptyset$ ;
2:   For  $k=1$  to 2
3:      $M \leftarrow M \cup Pop_k$ ;
4:   End For
5:    $M \leftarrow M \cup A$ ;
6:   For each solution  $x$  in  $A$ 
7:     Apply the mutation operator ( random mutation) to the variable  $x$ ;
8:      $M \leftarrow M \cup \{x\}$ ;
9:   End For
10:  Perform the duplication elimination process on  $M$  to get  $M'$ ;
11:   $A \leftarrow \emptyset$ ;
12:  If  $|M'|$  dose not exceed  $NA$ 
13:     $A \leftarrow M'$ ;
14:  Else
15:    Choose  $NA$  solutions from  $M'$  that have lower nondomination ranks and
    greater crowding distances and add them to archive  $A$ 
16:  End If
Output:  $A$  (updated archive)

```

3.7 Procedures of MPNSGA-II

Figure 7 illustrates the mechanism of our proposed MPNSGA-II. Initially, we initialize two separate populations aimed at optimizing two distinct objectives. Within each generation, the current population, denoted as pop_k for k in the range ($k \in [1, 2]$), collaborates with the archive A to generate crossover offspring. The offspring exhibiting superior performance on the k th objective is retained to form the new population pop'_k . Next, for each solution p'_k in pop'_k , a mutated solution c'_k is generated with a mutation probability determined by the mutation operator. The solution demonstrates a lower fitness about its corresponding optimization objective when compared to c'_i and p'_i is preserved in the mutated population pop''_k . Subsequently, Pop_k is updated with pop''_k . As a final step, the archive update strategy (AUS) is executed based on the newly updated populations Pop_k . The main process of MPNSGA-II, as shown in Fig. 7, includes multi-population genetic manipulation, updating the archive solution set, and other parts.

Algorithm 3 MPNSGA-II (Max_Gen, Pr_c , NA, N_Obj, N_Pop).

Input: Max_Gen, Pr_c , NA, N_Obj, N_Pop

```

1: Pop= Init_Populations (N_Obj, N_Pop);
2: A=Init_Archives (NA);
3: For G= 1 to Max_Gen
4:   For O= 1to N_Obj
5:     For i= 1 to N_Pop
6:       Child=ASS ( Pop, A, Prc );
7:       Mut= Mutation (Child);
8:       If Objectives [O] (Mut) ≤ Objectives [O] (Child):
9:         Child = Mut;
10:      End If
11:      Popk[i] = Child;
12:    End For
13:  End For
14:  A=AUS (Popk, A, NA);
15: End For
Output: A(Archive)

```

To sum it up, the MPMO framework ensures that each population effectively optimizes its unique objective. To prevent populations from exclusively concentrating on their respective goals, potentially causing underperformance in other areas, the ASS facilitates the exchange of information among populations through an archive. Furthermore, the AUS is specifically designed to enhance the quality of elite solutions, thereby guiding the evolution of all populations more effectively. Consequently, the MPNSGA-II mechanism can approximate the true Pareto front of MPNSGA-II by promoting cooperative coevolution.

Figure 8 provides a visual representation of the MPNSGA-II's flowchart. Initially, populations are created through random initialization. Subsequently, these populations are updated through the execution of crossover, ASS, and mutation

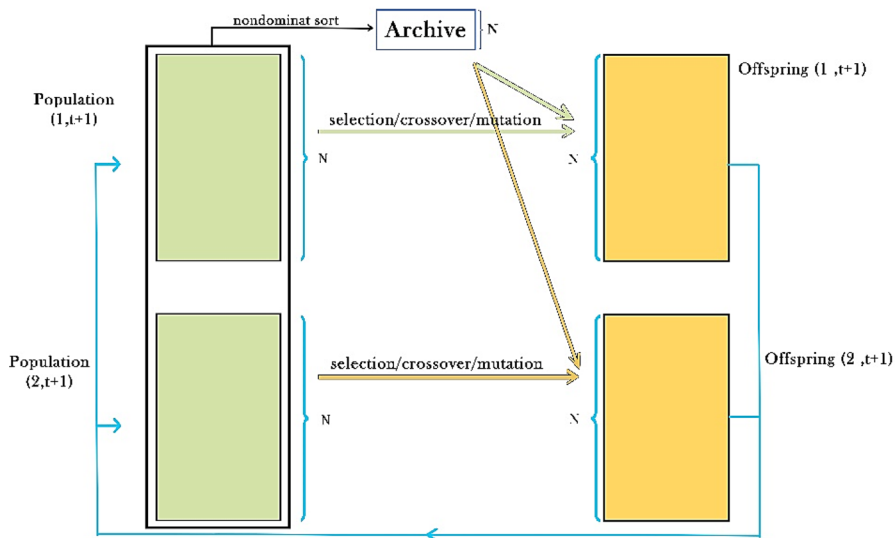


Fig. 7 Main procedure of the MPNSGA-II

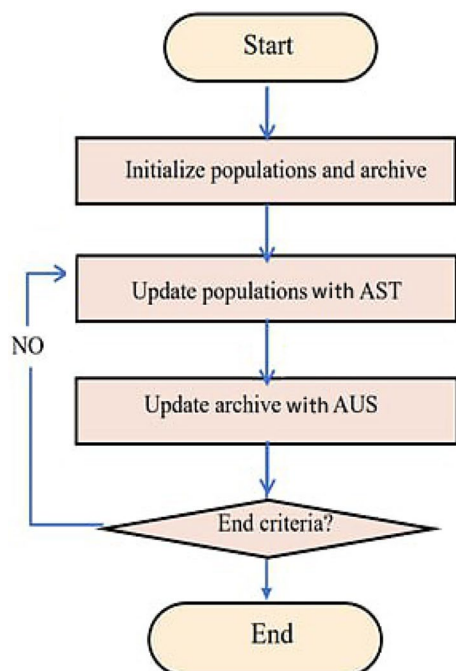
operators. Following this, archive A is updated using the AUS method. This evolutionary process continues iteratively until the predefined termination condition is met. The elite solutions stored within archive A serve as the ultimate output of the NSGA-II. Algorithm 3 presents the pseudocode for MPNSGA-II. MPNSGA-II incorporates an enhanced archive sharing strategy (ASS) to ensure a diverse set of elite solutions is maintained throughout the optimization process. MPNSGA-II's archive update strategy (AUS) dynamically adjusts the archive to balance convergence and diversity effectively, addressing this critical issue. Also this algorithm, is designed to handle large problem instances efficiently by optimizing computational complexity and leveraging parallel processing capabilities.

3.8 Computational complexity of MPNSGA-II

Considering the population size (PS), the dimension of each solution (D), and the maximum size of the archive (NA), where in MPNSGA-II, NA equals PS. The computational complexity of one generation of MPNSGA-II is primarily influenced by the genetic operators and the archive update strategy (AUS) aimed at enhancing elite solutions [42].

During the generation of offspring, the crossover operation, which produces PS crossover offspring each with D dimensions, requires $O(PS \times D)$ computations. Evaluating these new solutions takes $O(PS)$ computations. The mutation

Fig. 8 Flowchart of MPNSGA-II



operation also has a complexity of $O(PS \times D)$, with an additional $O(PS)$ computations needed for evaluating the mutated solutions. Thus, the total computational complexity for generating offspring using the single-point AST and IM operator is $O(2PS \times D + 2PS)$. In the AUS, local perturbation and solution evaluation require $O(NA \times D + NA)$ computations. The operation to eliminate duplicates involves pairwise comparisons, resulting in a complexity of $O(|S|^2)$. Since the maximum size of S is $PS + 2NA$ and NA equals PS , $O(|S|^2)$ simplifies to $O(PS^2)$. The complexity of the non-dominated sorting and diversity preservation methods is $O(PS^2)$, as noted in reference [43]. Consequently, the total computational complexity of the AUS is $O(NA \times D + NA + 2PS^2)$.

Overall complexity of MPNSGA-II, considering both the genetic operators and the AUS, is $O(3PS \times D + 3PS + 2PS^2)$, with NA substituted by PS . Simplified, the computational complexity for one generation of MPNSGA-II is $O(PS^2 + PS \times D)$.

4 Experimental results

4.1 Instance

Programming has been done with five small-scale scheduling cases and eight large-scale scheduling cases chosen from OR- Library (<http://people.brunel.ac.uk/~mastjjb/jeb/orlib/airlandinfo.html>) to validate the performance of the MPNSGA-II. These instances of benchmarks are frequently used in the literature [15, 44]. Our data includes 13 datasets, starting from “aircraft 1” which contains 10 aircraft, to “aircraft 13” which contains 500 aircraft.

4.2 Experimental setup

The evaluation of parameters involved the analysis of multiple tests and the determination of the algorithms’ optimal performance. The parameter configurations are presented in Table 2. The termination condition for all the tested algorithms is the maximum iteration. Throughout the experiments, Python was utilized to code all optimization algorithms and simulations, while the experimental setup was executed on a machine equipped with an Intel Core i7-6700HQ quad-core processor clocked at 2.60 GHz, 16 GB of RAM and running the Windows 10 operating system. For a fair comparison, all algorithms conduct 25 independent runs.

The experimental results of the proposed algorithm are compared to three effective multi-objective evolutionary algorithms (MOEAs) proposed for MOALP. Specifically, three multi-objective evolutionary algorithms (MOEAs) are: (1) Non-Dominated Sorting Genetic Algorithm-III (NSGA-III) [45]; (2) Non-Dominated Sorting Genetic Algorithm-II (NSGA-II) [43], and (3) the Multi-Objective Particle Swarm Optimization (MOPSO) [46]. As ALP is characterized as a discrete optimization problem, it is worth noting that the crossover and mutation operators of the three multi-objective evolutionary algorithms (MOEAs) were initially crafted to address continuous optimization challenges. Consequently, to render these MOEAs

Table 2 Parameter configurations

Parameter	Value	Description
Iter	2000	Iteration's quantity
nPop	400	Initial population size
nRep	100	Quantity of repositories in MOPSO
$\alpha 1$	0.1	Inflation lower rate in MOPSO
β	3	Leader selection pressure in MOPSO
γ	2	Deletion selection pressure in MOPSO
$\alpha 2$	0.9	Rate of Inflation upper in MOPSO
PC	0.9	Crossover Probability
PM	0.07	Mutation Probability

suitable for tackling ALP, we have undertaken adaptations by replacing their original crossover and mutation operators with single-point crossover and mutation techniques, respectively. This adjustment has been made to equip these algorithms with the capability to effectively address and solve the discrete nature of ALP. Note that the other methods remain the same as in their respective references.

To assess the convergence and diversity of the algorithms, the presented performance was evaluated with some of the famous metrics like IGD and HV. IGD is widely used because it captures both convergence and diversity. A lower IGD value indicates that the solution set is close to the true Pareto front and is well-distributed along it. HV is popular because it considers both the convergence of solutions towards the Pareto front and the spread of solutions along the front. A higher HV value indicates a better coverage of the Pareto front. Note that both the IGD and HV metrics undergo calculation in 25 separate runs. The results presented encompass the average outcome across these 25 runs. IGD measures the deviation from the ideal solutions to the Pareto front. HV quantifies the space occupied by dominated solutions concerning the optimal solutions and a reference point. In summary, superior algorithms are identified by smaller IGD values and higher HV values. The detailed mathematical expressions for these metrics can be referenced in [47]. A more effective front is associated with a higher HV, while lower values of IGD indicate better-performing fronts.

Before computing IGD and HV, the Pareto fronts (PFs) generated by the algorithms and the actual PF are standardized using equation:

$$\text{norm_}f_k(x) = \frac{f_k(x) - \text{PF}_{\min,k}}{\text{PF}_{\max,k} - \text{PF}_{\min,k}} \quad (12)$$

In the context of the notation used in Eq. 12 f_k , it signifies the fitness value associated with the k th objective, while $\text{norm_}f_k$ stands for the normalized fitness value corresponding to the k th objective. Additionally, $\text{PF}_{\max,k}$ $\text{PF}_{\min,k}$ denote the maximum and minimum values found on the actual Pareto Front for the k th objective, respectively. It is important to note that in real-world optimization problems, the true Pareto Front (PF) is typically unknown. As a result, the non-dominated solutions

extracted from the final results obtained by all the algorithms across 25 runs are considered the true Pareto Set (PS). The objectives associated with this true Pareto Set (PS) are then regarded as the authentic Pareto Front (PF).

The convergence evaluation of the algorithm is done through IGD metrics. It is shown by Eq. 13 where d is the Euclidean distance between Λ and P , assuming that $\Lambda = \{y_1, y_2, \dots, y_r\}$ is a close estimation of the true Pareto front and $P = \{p_1, p_2, \dots, p_k\}$ is the algorithm's outcome. While this metric shows the variation of the current solution from the Pareto front, lower IGDs show the convergence of the algorithm.

$$\text{IGD}(\Lambda, P) = \frac{1}{r} \left(\sum_{i=1}^r d(y_i, P)^2 \right)^{\frac{1}{2}} \quad (13)$$

HV metric, which calculates the objective space occupied by non-dominated solution sets considering the minimum objectives of a multi-objective optimization algorithm, is also one of the significant metrics for assessing multi-objective optimization algorithms. Higher HV could result in a more favorable performance of the algorithm and can be computed as:

$$\text{HV} = \text{Volume} \left(\bigcup_{i=1}^{|P_A|} V_i \right) \quad (14)$$

Moreover, the hypervolume (HV) metric quantifies the volume enclosed by a nadir point, and the normalized Pareto Front (PF) is achieved by the algorithm. A greater HV value signifies superior algorithmic performance.

4.3 Experimental results with participant algorithms

The assessment of solutions generated by MPNSGA-II is carried out by comparing them with competing algorithms, employing two distinct viewpoints. Firstly, the initial perspective entails an evaluation of the algorithm's performance through the utilization of metrics such as inverted IGD and HV. These metrics serve to offer a comprehensive assessment of the algorithm's efficacy in striking a harmonious equilibrium between the convergence and diversity of solutions. The second perspective concentrates on the scrutiny of the most optimal objectives attained throughout the evolutionary process. This analysis provides insights into the algorithm's capability to achieve superior outcomes and optimize each objective effectively.

4.3.1 Evaluation using IGD and HV metrics

The inverted generational distance (IGD) and hypervolume (HV) outcomes for all algorithms are documented in Table 3. It is important to observe that the results are presented in the average format, and the most superior results for each test instance

are highlighted in bold. Additionally, a comparison reveals that MPNSGA-II significantly outperforms better than NSGA-III, NSGA-II, and MOPSO.

As it is clear from Table 3, the MPNSGA-II presented in this article has been able to perform much better than other algorithms in the entire dataset. For example, in dataset 9, the MPNSGA-II performed much better than the NSGA-II algorithm, and it showed a significant decrease. Also, in datasets 10 and 11, the NSGA-II and NSGA-III algorithms performed close to each other, while the MPNSGA-II showed a significant decrease compared to other algorithms.

In comparison with other methods in objective trades, the current scenario indicated better performance by offering low values for either objective, as shown in Table 3. Due to considerably lower IGD, the presented algorithm, in comparison with the other three, shows better convergence. Because of its higher HV metric, which shows the efficiency of an algorithm, the presented method is proven to be more efficacious with diverse Pareto front points. According to Table 3, the MOPSO algorithm offers higher objective values as well as lower diversity and convergence compared to others. Furthermore, MPNSGA-II shows its superiority through the test.

The application of a normalization method effectively confines the real Pareto Front (PF) distribution within the $[0, 1]$ range. It is worth noting that a higher degree of inverted generational distance (IGD) signifies a lower level of convergence for the

Table 3 Results between MPNSGA-II and the competitor algorithms on large-scale instances

Dataset/Algorithm	First objective (Cost)	Second objective (Dwell Time)
Airland 9/NSGA-II	23,541.2	23,600
Airland 9/NSGA-III	23,439.1	23,580.12
Airland 9/MOPSO	21,472.94	24,012
Airland 9/MPNSGA-II	13,980.41	23,412.1
Airland 10/NSGA-II	39,275.1	40,520
Airland 10/NSGA-III	39,340.4	40,580.6
Airland 10/MOPSO	37,942.94	41,043.1
Airland 10/MPNSGA-II	29,800.3	39,410.1
Airland 11/NSGA-II	42,700.13	44,295.1
Airland 11/NSGA-III	42,691.24	44,450.12
Airland 11/MOPSO	40,471.22	44,003.1
Airland 11/MPNSGA-II	30,051.1	43,025.1
Airland 12/NSGA-II	59,800	52,350
Airland 12/NSGA-III	59,850	52,400
Airland 12/MOPSO	57,500	52,000
Airland 12/MPNSGA-II	44,000	51,000
Airland 13/NSGA-II	108,900	119,800
Airland 13/NSGA-III	109,080	120,075
Airland 13/MOPSO	107,000	119,500
Airland 13/MPNSGA-II	92,500	115,000

algorithm. Furthermore, increased diversity indicates a broader range of PF distribution achieved by the algorithm, suggesting enhanced algorithmic performance in this aspect.

4.3.2 Assessment of objective optimization efficiency

Table 3 presents a comprehensive overview of algorithm performance concerning the optimization of individual objectives. The findings reveal that all algorithms perform admirably when dealing with smaller aircraft dimensions, successfully optimizing both objectives. However, in the case of larger dimensions, the algorithms exhibit suboptimal performance. Notably, the presented algorithm stands out by achieving complete optimization of the second objective while minimizing the first objective as effectively as possible. To facilitate a meaningful comparison of the performance of the different algorithms under investigation, we have employed various algorithms to generate Pareto fronts (PFs) for the specified test case. Figures 9 and 10 depict the PFs produced by all the tested algorithms for the Aircraft 12 and Aircraft 13 scenarios.

Figure 9 shows the difference between the presented algorithm and the other algorithms for dataset 12 with 250 aircraft. As it is clear from Fig. 9a, compared to the existing methods, the proposed method has been able to achieve a very good reduction in the first goal, i.e., cost, and the second goal, i.e., time to complete the

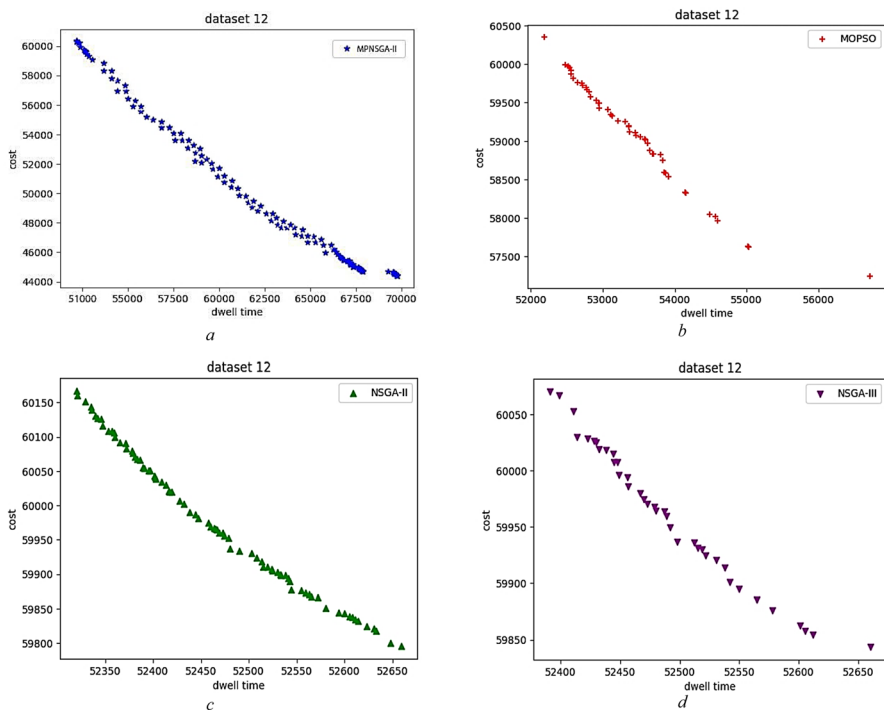


Fig. 9 Proposed method is in comparison with the competitor algorithm For 250 Aircraft

work. Also, according to Fig. 9c and d, NSGA-II and NSGA-III have almost similar performance, which has been able to reduce both objectives to almost the same amount.

Figure 10 shows the results of four different algorithms used to decrease both dwell time and cost. Figure 10a represents the results of using multiple populations for multiple objectives-based NSGA-II (MPNSGA-II), which significantly decreases both dwell time and cost. Figure 10b shows the results of using the MOPSO algorithm, which also has good results but does not perform much better than our provided method. Figure 10c and d depict the results of using NSGA-II and NSGA-III, both non-dominated sorting genetic algorithms. However, these algorithms do not improve either the cost or dwell time compared to MPNSGA-II and MOPSO. In conclusion, MPNSGA-II performs the best, reducing the cost to 92,500 units and dwell time to 115,000, followed by the MOPSO algorithm as the second-best performer.

As it is clear from Table 3, the MPNSGA-II presented in this article has been able to perform much better than other algorithms in the entire dataset. For example, in dataset 9, the MPNSGA-II performed much better than the NSGA-II algorithm, and it showed a significant decrease. Also, in datasets 10 and 11, the NSGA-II and NSGA-III algorithms performed close to each other, while the MPNSGA-II showed a significant decrease compared to other algorithms.

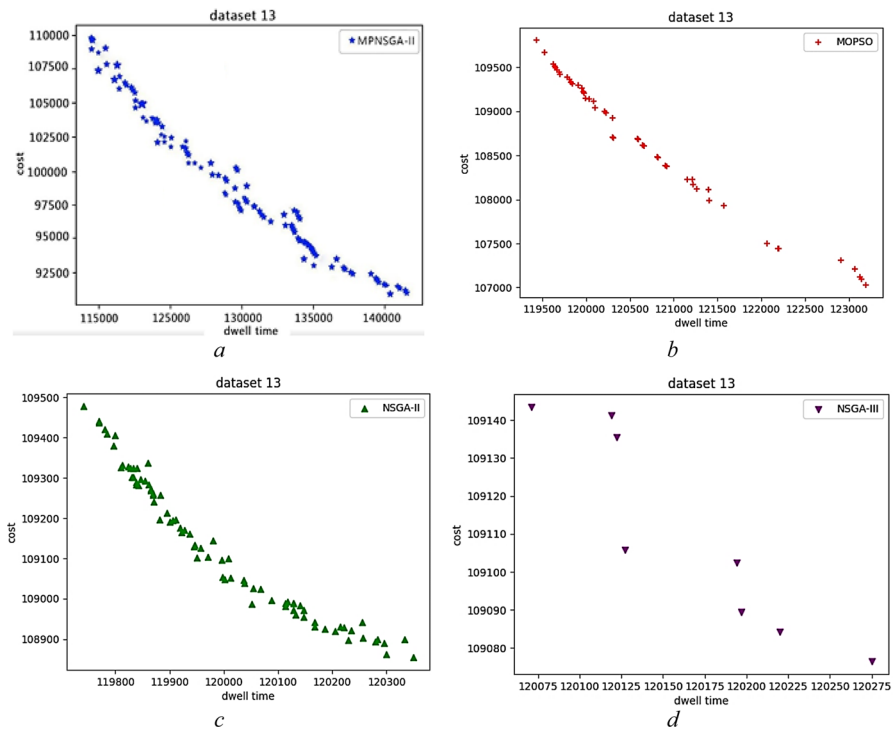


Fig. 10 Proposed method is compared to competitor algorithms for 500 aircraft

In comparison with other methods in objective trades, the current scenario indicated better performance by offering low values for either objective, as shown in Table 4. Due to considerably lower IGD, the presented algorithm, in comparison with the other three, shows better convergence. Because of its higher HV metric, which shows the efficiency of an algorithm, the presented method is proven to be more efficacious with diverse Pareto front points. According to Table 4, the MOPSO algorithm offers higher objective values as well as lower diversity and convergence compared to others. The most superior results for each instance are highlighted in bold. Furthermore, MPNSGA-II shows its superiority through the test.

Due to considerably lower IGD, the presented algorithm, in comparison with the other three, shows better convergence. Because of its higher HV metric, which shows the efficiency of an algorithm, the presented method is proven to be more efficacious with diverse Pareto front points. According to Table 4, the MOPSO algorithm offers higher objective values as well as lower diversity and convergence compared to others. Furthermore, MPNSGA-II shows its superiority through the test. As shown in Table 5, our proposed method demonstrates enhanced convergence capabilities when compared to these aforementioned methods. The analysis of the results shows that the algorithm presented in this study has a better performance in terms of dwell time compared to the algorithms NSGA-II, NSGA-III, and MOPSO, with improvements of 4.41%, 4.17%, and 3.91%, respectively. Moreover, the proposed algorithm shows a reduction in cost of 18.01%, 16.75%, and 15.21% compared to

Table 4 Metric results between MPNSGA-II and the competitor algorithms on large-scale instances

Algorithms/dataset	IGD	HV
NSGA-II/dataset9	0.962	0.372
NSGA-III/dataset9	0.952	0.475
MOPSO/dataset9	0.828	0.211
MPNSGA-II/dataset9	0.609	0.564
NSGA-II/dataset10	0.97453	0.02539
NSGA-III/dataset10	0.9809	0.0196
MOPSO/dataset10	0.8714	0.14928
MPNSGA-II/dataset10	0.578659	0.5833
NSGA-II/dataset11	0.95249	0.475
NSGA-III/dataset11	0.9536	0.463
MOPSO/dataset11	0.828	0.2045
MPNSGA-II/dataset11	0.575	0.602
NSGA-II/dataset12	0.95664	0.043
NSGA-III/dataset12	0.96	0.395
MOPSO/dataset12	0.814958	0.21083
MPNSGA-II/dataset12	0.514	0.6537
NSGA-II/dataset13	0.950	0.4985
NSGA-III/dataset13	0.961	0.379
MOPSO/dataset13	0.8686	0.137
MPNSGA-II/dataset13	0.608	0.584

Table 5 Multi-objective optimization algorithms result in Airland13

Optimization algorithm	MPNSGA-II	MOPSO	NSGA-II	NSGA-III
Max dwell time	142,400	123,012	120,359	120,275
Min dwell time	115,000	119,500	119,800	120,075
Max cost	110,000	109,580	109,480	109,200
Min cost	92,500	107,000	108,000	109,080
IGD	0.608	0.8686	0.950	0.961
HV	0.584	0.137	0.04985	0.0379

the aforementioned algorithms. The most superior results for each test instance are highlighted in bold.

4.4 Statistical analysis

Friedman and post hoc tests as nonparametric tests were employed to prove the reliability of the comparison done in this research. According to Friedman, unless the null hypothesis gets rejected, test algorithms are alike [48]. Figures 11 and 12 illustrate the Friedman test results computed from four different algorithms to identify the differences in the results.

As shown in Fig. 11, the Friedman test ranked the algorithms according to their performance. MPNSGA-II has proven to be the best in all results and showed a meaningful difference from the other three algorithms according to experiments and metrics. Tables 3 and 4 indicate the results referred to IGD and HV, respectively. Moreover, since the value of p is about zero, the null hypothesis for all tests can be rejected, which leads to a great difference in the MPNSGA-II performance.

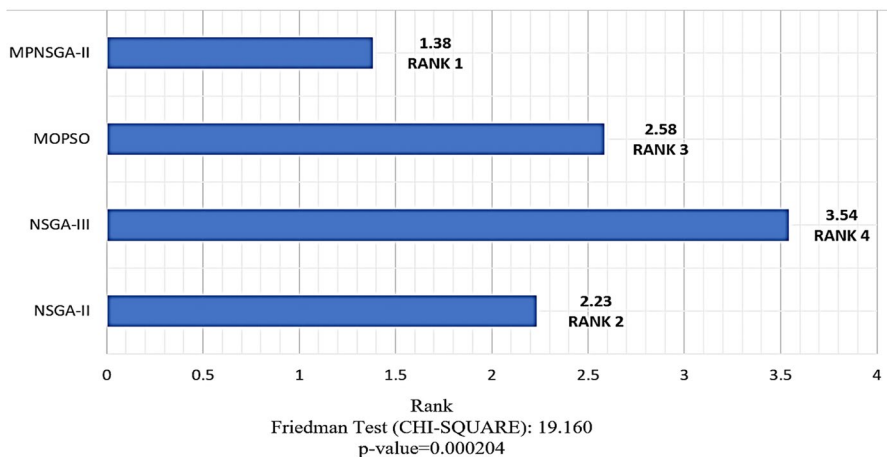


Fig. 11 Average ranks obtained by each experimental result and metric IGD in the Friedman test

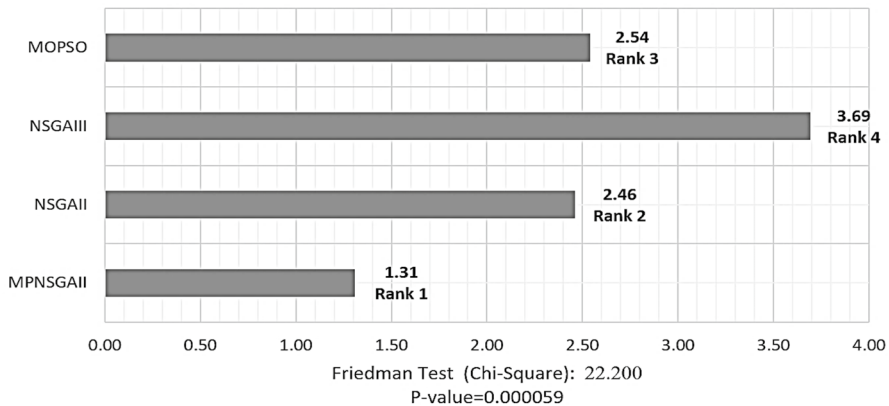


Fig. 12 Average ranks obtained by each experimental result and metric HV in the Friedman test

The post hoc test is a statistical method used to compare multiple treatments (or algorithms, in this context) simultaneously. It is particularly useful when you have several algorithms and aim to determine if there are significant differences in their performance across multiple datasets or scenarios. As mentioned, post hoc analysis is essential for pairwise comparisons. Therefore, we compared the proposed algorithm in this paper with three other algorithms using the post hoc method, and the results are presented in Tables 6 and 7. The post hoc analysis was conducted using the Wilcoxon signed-rank tests with a Bonferroni correction applied, resulting in a significance level set at $p < 0.05$. As shown in Tables 6 and 7, there were significant differences between all algorithms compared to MPNSGA-II. Thus, MPNSGA-II outperforms the other algorithms.

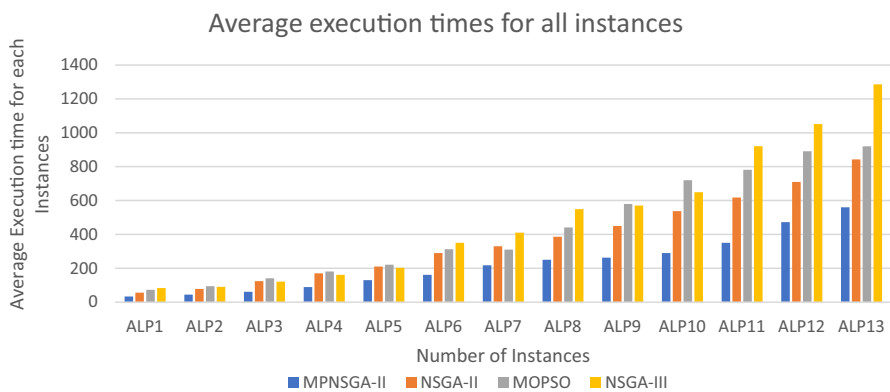
Figure 13 demonstrates the scheduling methods for all aircraft landing instances. While all methods show the same results for small-scale schedulings, MPNSGA-II demonstrates incredibly better performance and faster convergence compared to NSGA-II, NSGA-III, and MOPSO when the number of aircraft increases to the range of 100–500.

Table 6 Post hoc statistical analysis results: IGD pairwise comparisons

Comparison	<i>P</i> value	<i>P</i> value (Bonferroni)
MOPSO, NSGA-II	≈ 0 (5.67×10^{-11})	≈ 0 (3.40×10^{-10})
MOPSO, NSGA-III	≈ 0 (6.25×10^{-6})	≈ 0 (3.75×10^{-5})
MOPSO, MPNSGA-II	≈ 0 (4.85×10^{-6})	≈ 0 (2.91×10^{-5})
MPNSGA-II, NSGA-II	≈ 0 (2.02×10^{-13})	≈ 0 (1.21×10^{-12})
MPNSGA-II, NSGA-III	≈ 0 (1.83×10^{-13})	≈ 0 (1.10×10^{-12})
NSGA-II, NSGA-III	≈ 0 (0.0009)	≈ 0 (0.00012)

Table 7 Post hoc statistical analysis results: HV pairwise comparisons

Comparison	<i>P</i> value	<i>P</i> value (Bonferroni)
MOPSO, NSGA-II	0.00257	0.00154
MOPSO, NSGA-III	0.00724	0.0031
MOPSO, MPNSGA-II	0.00349	0.00424
MPNSGA-II, NSGA-II	0.0215	0.0129
MPNSGA-II, NSGA-III	0.0076	0.00467
NSGA-II, NSGA-III	0.000109	0.00012

**Fig. 13** Average execution times for all instances**Table 8** Three factors (control parameters) and four levels (parameter values)

Factors	Level1	Level2	Level3	Level4
Population size	100	200	300	400
Crossover rate	0.6	0.7	0.8	0.9
Mutation rate	0.2	0.1	0.06	0.05

4.5 Sensitivity analysis

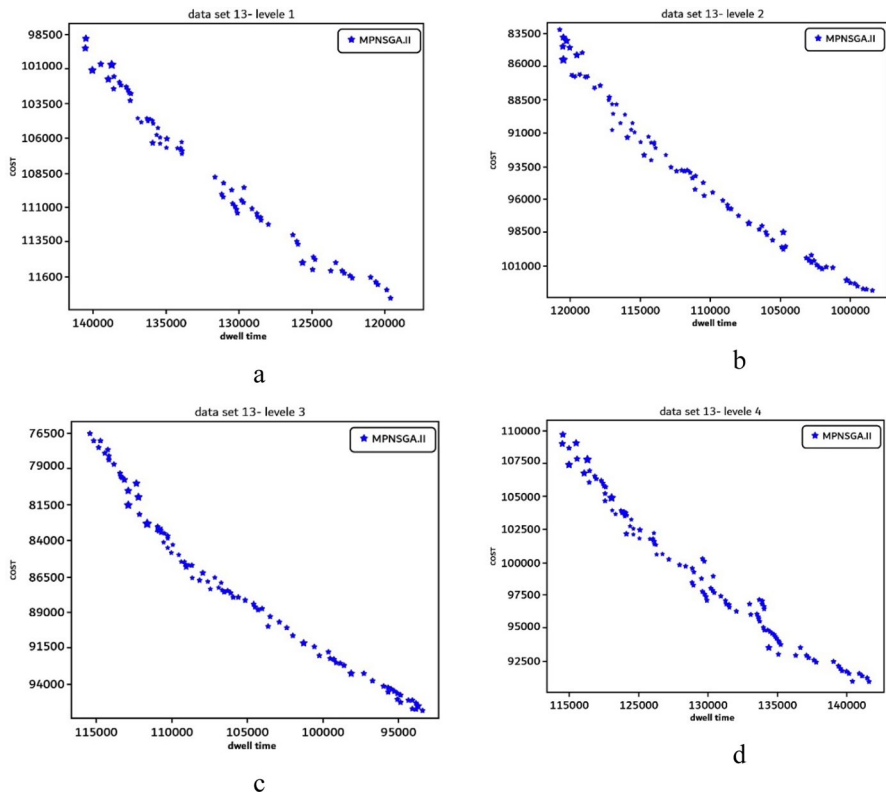
To address concerns regarding the sensitivity of the algorithm to various parameters, a comprehensive sensitivity analysis was conducted. This analysis aimed to understand how different parameter settings influence the algorithm's performance and to guide the selection of appropriate values for diverse problem instances.

The sensitivity analysis focused on three key parameters: (i) Population size, (ii) Crossover rate, and (iii) Mutation rate. The parameters were varied within the following ranges to explore their impact on algorithm performance:

The performance of the algorithm was evaluated based on two objectives: Objective 1 (cost) and Objective 2 (dwell time) (Table 8). The results from the sensitivity analysis are summarized in Tables 9.

Table 9 Results of sensitive analysis in different parameters

Levels	Objective 1 (cost)	Objective 2 (dwell time)
Level 1	116,419	140,131
Level 2	101,300	121,081
Level 3	94,290	116,400
Level 4	92,500	115,000

**Fig. 14** Sensitivity of the algorithm to various parameters

Based on the sensitivity analysis, the following conclusions and recommendations can be made: A population size of 400, a crossover rate of 0.9, and a mutation rate of 0.05 consistently yielded the best results across both cost and dwell time objectives. These settings demonstrated superior performance in optimizing the algorithm's output. The algorithm exhibited robustness in maintaining competitive performance across a range of mutation rates. However, it was observed to be more sensitive to changes in crossover rates, where higher values generally provided better

exploration capabilities. Recommendations for parameter selection include utilizing higher crossover rates (e.g., 0.8 or 0.9) for tasks requiring extensive exploration of the search space. Conversely, lower crossover rates (e.g., 0.6 or 0.7) may be more suitable for tasks focused on exploiting known solutions.

As Fig. 14 shows, the changes in the parameters of the algorithm have a significant effect on the production values in both objectives.

5 Conclusion

This article introduces a novel MOALP model with dual objectives focused on minimizing both completion time (dwell time) and total tardiness to achieve cost reduction. To effectively address this model, we have devised a new algorithm named MPNSGA-II, which is built upon the MPMO framework. The MPMO framework's first goal is to employ separate populations for the concurrent optimization of various objectives. Additionally, in this paper, we present the concept of ASS within MPNSGA-II to facilitate the coevolution of all populations, and it introduces the AUS, a feature designed to improve the quality of the elite solutions stored in the archive. The performance of our proposed MPNSGA-II is assessed using a commonly employed test set. The experimental outcomes indicate that MPNSGA-II performs admirably, excelling in both widely recognized metrics, namely IGD and HV, and also achieving optimal values for each objective. The experimental analysis underscores the effectiveness of the MPMO framework in addressing MOALP. Additionally, we delve into the impact of perturbation in the AUS and how it enhances the overall performance of MPNSGA-II. Statistical analyses demonstrate that the proposed method achieves the highest ranking in most of the tests. This problem has been solved in a single runway, and in future work, we intend to solve this problem in multi-runway mode by considering the problem dynamically.

Author contribution Kimia Shirini involved in conceptualization, data curation, formal analysis, investigation, methodology, software, validation, visualization, and writing—original draft. Hadi S. Aghdasi involved in conceptualization, data curation, formal analysis, investigation, methodology, project administration, supervision, software, validation, visualization, and writing—review and editing. Saeed Saeedvand involved in conceptualization, formal analysis, investigation, methodology, validation, and writing—review and editing.

Funding No funding was obtained for this study.

Data availability All datasets are open source, and the sources are cited. Data are available upon request.

Declarations

Conflict of interest The authors declare no competing interests.

Ethics approval This paper has never been submitted or introduced to any journals or conferences before.

References

1. Kursat Cecen R, Saraç T, Cetek C (2023) Emission and flight time optimization model for aircraft landing problem. *Transp Res Record* 2677(2):763–773
2. Silva LC et al (2023) A new modelling for the aircraft landing problem and matheuristic approach to solve the problem with a large number of aircraft. *Pesquisa Operacional* 43:e266290
3. Messaoud MB (2021) A thorough review of aircraft landing operation from practical and theoretical standpoints at an airport which may include a single or multiple runways. *Appl Soft Comput* 98:106853
4. Su J et al (2023) A large neighborhood search algorithm with simulated annealing and time decomposition strategy for the aircraft runway scheduling problem. *Aerospace* 10(2):177
5. Nikolić M, Rakas J, Teodorović D (2024) Solving the aircraft landing problem using the bee colony optimization (BCO) algorithm
6. Shirini K, Aghdasi HS, Saeedvand S (2024) Modified imperialist competitive algorithm for aircraft landing scheduling problem. *J Supercomput* p. 1–31
7. Aryania A, Aghdasi HS, Khanli LM (2018) Energy-aware virtual machine consolidation algorithm based on ant colony system. *J Grid Comput* 16:477–491
8. Mirjalili S, (2019) Genetic algorithm. *Evolutionary algorithms and neural networks: Theory and applications* p. 43–55
9. Mirjalili S, Song Dong J, Lewis A (2020) Ant colony optimizer: theory, literature review, and application in AUV path planning. *Nature-inspired optimizers: theories, literature reviews and applications* p. 7–21
10. Nssibi M et al (2024) Gene selection for high dimensional biological datasets using hybrid island binary artificial bee colony with chaos game optimization. *Artif Intell Rev* 57(3):1–74
11. Beasley JE et al (2000) Scheduling aircraft landings—the static case. *Transp Sci* 34(2):180–197
12. Vadamani S, Hosseini S (2014) A novel heuristic approach for solving aircraft landing problem with single runway. *J Air Transp Manag* 40:144–148
13. Lieder A, Briskorn D, Stollitz R (2015) A dynamic programming approach for the aircraft landing problem with aircraft classes. *Eur J Oper Res* 243(1):61–69
14. Yua S, Cao X, Zhang J (2011) A realYtime schedule method for aircraft landing scheduling problem based on cellular automaton. *Appl Soft Comput* 11:3485–3493
15. Salehipour A, Modarres M, Naeni LM (2013) An efficient hybrid meta-heuristic for aircraft landing problem. *Comput Oper Res* 40(1):207–213
16. Girish B (2016) An efficient hybrid particle swarm optimization algorithm in a rolling horizon framework for the aircraft landing problem. *Appl Soft Comput* 44:200–221
17. Faye A (2015) Solving the aircraft landing problem with time discretization approach. *Eur J Oper Res* 242(3):1028–1038
18. Sabar NR, Kendall G (2015) An iterated local search with multiple perturbation operators and time varying perturbation strength for the aircraft landing problem. *Omega* 56:88–98
19. Ji X-P, Cao X-B, Tang K (2016) Sequence searching and evaluation: a unified approach for aircraft arrival sequencing and scheduling problems. *Memetic Comput* 8:109–123
20. Hu X-B, Chen W-H (2005) Receding horizon control for aircraft arrival sequencing and scheduling. *IEEE Trans Intell Transp Syst* 6(2):189–197
21. Hu X-B, Di Paolo E (2008) Binary-representation-based genetic algorithm for aircraft arrival sequencing and scheduling. *IEEE Trans Intell Transp Syst* 9(2):301–310
22. Zhan Z-H et al (2010) An efficient ant colony system based on receding horizon control for the aircraft arrival sequencing and scheduling problem. *IEEE Trans Intell Transp Syst* 11(2):399–412
23. Eun Y, Hwang I, Bang H (2010) Optimal arrival flight sequencing and scheduling using discrete airborne delays. *IEEE Trans Intell Transp Syst* 11(2):359–373
24. Harikiopoulou D, Neogi N (2010) Polynomial-time feasibility condition for multiclass aircraft sequencing on a single-runway airport. *IEEE Trans Intell Transp Syst* 12(1):2–14
25. Ji X-P et al (2017) An evolutionary approach for dynamic single-runway arrival sequencing and scheduling problem. *Soft Comput* 21:7021–7037
26. Tang K et al. (2008) A multi-objective evolutionary approach to aircraft landing scheduling problems. In: 2008 IEEE Congress on Evolutionary Computation (IEEE World Congress on Computational Intelligence). IEEE

27. Hong Y et al (2017) Multiobjective optimization for aircraft arrival sequencing and scheduling. *J Air Transp* 25(4):115–122
28. Shirini K, Aghdasi HS, Saeedvand S (2024) A comprehensive survey on multiple-runway aircraft landing optimization problem. *Int J Aeronaut Space Sci (IJASS)*
29. Ghasemzadeh A, Aghdasi HS, Saeedvand S (2024) Edge server placement and allocation optimization: a tradeoff for enhanced performance. *Cluster Comput* p 1–15
30. Beasley JE, Sonander J, Havelock P (2001) Scheduling aircraft landings at London Heathrow using a population heuristic. *J Operat Res Soc* 52(5):483–493
31. Pholdee N, Bureerat S (2014) Comparative performance of meta-heuristic algorithms for mass minimisation of trusses with dynamic constraints. *Adv Eng Softw* 75:1–13
32. Bennell JA, Mesgarpour M, Potts CN (2017) Dynamic scheduling of aircraft landings. *Eur J Oper Res* 258(1):315–327
33. Faye A (2018) A quadratic time algorithm for computing the optimal landing times of a fixed sequence of planes. *Eur J Oper Res* 270(3):1148–1157
34. Malik W, Lee H, Jung YC (2016) Runway scheduling for Charlotte Douglas international airport. In 16th AIAA Aviation Technology, Integration, and Operations Conference
35. Pohl M, Kolisch R, Schiffer M (2021) Runway scheduling during winter operations. *Omega* 102:102325
36. Ahmed MS, Alam S, Barlow M (2018) A cooperative co-evolutionary optimisation model for best-fit aircraft sequence and feasible runway configuration in a multi-runway airport. *Aerospace* 5(3):85
37. Hammouri AI et al (2020) ISA: a hybridization between iterated local search and simulated annealing for multiple-runway aircraft landing problem. *Neural Comput Appl* 32:11745–11765
38. Ebrahim RM, Razmi J (2009) A hybrid meta heuristic algorithm for bi-objective minimum cost flow (BMCF) problem. *Adv Eng Softw* 40(10):1056–1062
39. Salehipour A (2017) A heuristic algorithm for the aircraft landing problem. In: 22nd International congress on modelling and simulation. Modelling and Simulation Society of Australia and New Zealand Inc.(MSSANZ)
40. Zhan Z-H et al (2013) Multiple populations for multiple objectives: A coevolutionary technique for solving multiobjective optimization problems. *IEEE Trans Cybern* 43(2):445–463
41. Kersting P, Zabel A (2009) Optimizing NC-tool paths for simultaneous five-axis milling based on multi-population multi-objective evolutionary algorithms. *Adv Eng Softw* 40(6):452–463
42. Liu S-C et al (2021) Many-objective job-shop scheduling: a multiple populations for multiple objectives-based genetic algorithm approach. *IEEE Trans Cybern* 53(3):1460–1474
43. Deb K et al (2002) A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans Evol Comput* 6(2):182–197
44. Pinol H, Beasley JE (2006) Scatter search and bionomic algorithms for the aircraft landing problem. *Eur J Oper Res* 171(2):439–462
45. Deb K, Jain H (2013) An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: solving problems with box constraints. *IEEE Trans Evol Comput* 18(4):577–601
46. Martínez-Cagigal V (2020) Multi-objective particle swarm optimization (MOPSO). *MATLAB Central File Exchange* p. 5–14
47. Panagant N et al. (2021) A comparative study of recent multi-objective metaheuristics for solving constrained truss optimisation problems. *Arch Comput Methods Eng* p 1–17
48. Saeedvand S, Aghdasi HS, Baltes J (2020) Novel hybrid algorithm for team orienteering problem with time windows for rescue applications. *Appl Soft Comput* 96:106700

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Kimia Shirini¹ · Hadi S. Aghdasi¹ · Saeed Saeedvand²

✉ Hadi S. Aghdasi
aghdasi@tabrizu.ac.ir

Kimia Shirini
k.shirini@tabrizu.ac.ir

Saeed Saeedvand
saeedvand@ntnu.edu.tw

¹ Faculty of Electrical and Computer Engineering, University of Tabriz, Tabriz, Iran

² Department of Electrical Engineering, National Taiwan Normal University, Taipei, Taiwan